A Glass Box Around the Black Box: Deterministic Auditing of LLM Answers Without Inference

Karthik Barma*
MS Artificial Intelligence
Khoury College of Computer Sciences
Northeastern University
barma.k@northeastern.edu

Sarita Singh
Associate Teaching Professor
Khoury College of Computer Sciences
Northeastern University
s.singh@northeastern.edu

Abstract

Tools that audit AI answers generally assume a model in the loop: a judge model, a classifier, or a guardrail service carrying its own inference cost. That assumption makes auditing something a deployment enables deliberately, for a subset of traffic, rather than something that can be on by default. We ask how much useful auditing survives when the auditor is permitted no model inference at all.

We present GlassBox Lite, a deterministic structural auditor that examines an explicitly submitted question and answer pair using seven bounded probes: claim segmentation, allowlisted arithmetic recomputation, direct lexical contradiction, unsupported absolute certainty, citation transparency, external fact-check scope, and prompt-injection signals. It issues no model call and performs no network lookup. It never inspects the generating model, so it places a glass box *around* a black box rather than opening one, and it emits a compact Trust Card carrying a verdict, a structural score, per-probe outcomes, and explicit limitations.

Removing inference has a consequence beyond cost. Because identical inputs yield identical outputs, determinism becomes a *deployment* property as much as an evaluation one, making idempotency, replay suppression, and reproducible audit records exact rather than approximate. We confirmed this against the live public endpoint, where three identical production requests returned byte-identical responses.

We then operate this auditor behind eight heterogeneous surfaces and report what deployment actually costs. The public Trust Card projection withholds submitted content and every verifier-internal field, verified by planting sentinel tokens in a live request and confirming their absence. All provider routes fail closed on unsigned requests. At the evaluated commit, 108 automated tests pass across six suites and twelve continuous-integration jobs succeed on the merged main commit.

Our most transferable contribution is methodological. Deployment claims in this area routinely collapse five distinct states into the word "available." We separate implementation presence, provider registration, public installability, production end-to-end evidence, and directory approval into an explicit five-level scale, and report our own results against it, including the surfaces that sit low on it.

We then measure the probes themselves on a constructed benchmark of 187 items with labels fixed by construction. Precision is 1.000 on every probe with zero false positives, but recall splits the probes in two: those that *compute* a property generalise to unseen phrasing (arithmetic recomputation 1.000, contradiction 0.833) while those that *match a vocabulary* do not (0.000 to 0.333 on a held-out split). Repairing the vocabularies raised development recall to 1.000 and moved held-out recall almost not at all. That is a bound on the zero-inference design, not a defect list. GlassBox remains a structural auditor and never a fact-checker: we measure whether probes fire correctly, not whether answers are true.

1 Introduction

An answer auditor that costs money per audit is an auditor that gets switched off. If verification requires a paid model API, the verification budget scales with traffic, and the rational deployment response is to sample: audit some answers, some of the time, in some contexts. The cases most worth auditing then become the ones most likely to be missed.

This paper explores the opposite corner of the design space. We constrain the auditor to perform no model inference whatsoever and ask what remains. The honest answer is: less than a model-based judge [22], but more than nothing, and what remains has properties a model-based judge cannot have.

What remains is *structural* auditing. GlassBox Lite does not decide whether an answer is true. It decides whether an answer is internally well formed: whether its arithmetic recomputes, whether it contradicts itself, whether it asserts certainty it has not earned, whether it cites anything, and whether it contains instruction-like text that a downstream client should treat as inert. These properties are narrow. They are also checkable without a model, and checkable identically every time.

The metaphor in our title is deliberate and load-bearing. Interpretability research opens the black box, and glass-box

*First author. GlassBox is developed under *Aura*, an unregistered umbrella brand and not a company or legal entity. The system charges no fee, accepts no payments, and sells nothing.

modelling replaces it with an inherently transparent one. We do neither. GlassBox never inspects weights, activations, or model internals, and it does not require the model to explain itself, which is fortunate given the evidence that such explanations can be unfaithful [21] and that model self-report is a fragile audit substrate [11]. It places a transparent enclosure around an opaque generator: transparency about the *answer*, with zero access to the *model*.

That constraint is where the paper turns. A deterministic auditor is not merely cheaper than a sampled one, it is operationally different. Retry handling becomes exact rather than probabilistic. A duplicate provider delivery can be suppressed with confidence that the suppressed result would have been identical. An audit record can be reproduced from its inputs. None of this is available to a service whose verdict is drawn from a sampling distribution, which is the position of sampling-based detectors such as SelfCheckGPT [16].

We therefore treat deployment as the primary object of study rather than an afterthought. GlassBox runs as a single verification service behind a web application, a remote Model Context Protocol (MCP) endpoint [17], a GitHub App, Discord, Telegram, Slack, an authenticated API, a browser extension, two IDE clients, a Notion connection, and a Reddit Devvit app. Building the verifier took a fraction of the effort. Satisfying eight providers' authentication schemes, invocation models, consent requirements, acknowledgement deadlines, and review processes took the rest.

That asymmetry produced our second contribution, a reporting discipline. Partway through this work it became clear that "GlassBox is available on Discord" and "GlassBox is available on GitHub" were both true sentences describing very different situations, and that no vocabulary distinguished them. We introduce a five-level evidence scale (Section 9) and hold every claim to it, including the unflattering ones.

Contributions.

1. A deterministic, zero-inference structural auditor with seven bounded, explainable probes, demonstrated byte-identical on a live public endpoint.
2. A privacy-minimized public Trust Card projection that withholds submitted content and all verifier-internal metadata, verified by planted-token testing against production.
3. A cross-platform gateway preserving provider-native authentication, explicit invocation, tenant admission, idempotency, rate limits, deadlines, and output neutralization, with every provider route verified to fail closed.
4. A probe-level benchmark with a held-out split, yielding the finding that model-free probes generalise exactly when they compute a property and barely at all when they match a lexicon.
5. Reproducibility evidence at a fixed public commit, and a five-level evidence discipline under which negative results are reported as first-class findings.

Non-goals. GlassBox is not a fact-checker, a source authenticator, a truth oracle, a hallucination detector with measured accuracy, a semantic contradiction engine, a general-purpose calculator, or a complete prompt-injection security boundary. The precision and recall reported in Section 11 describe whether each probe fires when it should on a constructed benchmark; they say nothing about how often these failure modes occur in real traffic, and nothing about factual truth. Section 17 states these limits in full.

2 Problem Definition

Consider an answer containing the expression $9 \times 9 = 80$. Detecting this requires no world knowledge, no retrieval, and no model. It requires parsing an arithmetic expression and recomputing it. Yet in most deployed architectures, catching it means either a second model call or nothing at all.

Structural errors of this kind form a real and under-served class. An answer may contradict itself between its second and fifth sentences. It may assert that something is "certainly" true while supplying no support. It may reference sources that do not exist as citations at all. It may contain text engineered to be read as an instruction by whatever consumes it [7, 18]. None of these require establishing external truth, and all are visible in the structure of the answer itself. This is a deliberately narrower target than the hallucination problem as usually framed [10, 8], and the narrowing is what makes a model-free treatment possible.

We define the problem accordingly. *Given an explicitly submitted question and answer pair, produce a bounded, deterministic, explainable structural audit without model inference, and deliver it across heterogeneous platforms without leaking the submitted content.*

Three constraints shape everything that follows. **Zero paid API:** the verifier may not call a paid model, which is what makes default-on auditing economically possible and what forces the probes to be conservative. **Opt-in only:** GlassBox never audits ambient traffic, it runs when a user explicitly invokes it. **Privacy-minimized output:** on a public interface the result must not echo the submitted content nor expose verifier internals, because the audit is useful precisely to the extent it can be shown to someone without re-disclosing what was audited.

3 Threat Model

We consider five adversaries. A **hostile submitter** supplies content designed to manipulate the auditor or the downstream client: prompt injection, hostile markup, control and bidirectional characters, oversized payloads. A **forger** sends unsigned or wrongly signed provider webhooks. A **replay adversary** resends valid provider deliveries to duplicate work. A **resource adversary** floods the service to exhaust a free-tier budget. A **curious observer** reads a public Trust Card hoping to recover the submitted content or the verifier's internal reasoning.

Controls appear in Section 8 and their verification in Section 10. These are implementation-tested controls, not a security assessment. We have conducted no penetration test,

no static analysis at scale, and no third-party audit, and we claim no compliance certification.

Out of scope: a malicious platform provider, a compromised host, network adversaries beyond TLS, and any claim about the correctness of the audited answer's external facts.

4 Trust Cards and the Structural Score

A Trust Card is the unit of output: a verdict, a structural score, a claim count, a finding count, a highest-severity marker, per-probe outcomes, and a caveat list.

The score is an Epistemic Confidence Score (ECS), and its name is the most dangerous thing about it. **ECS is a structural score. It is not a probability that the answer is true, and it is not calibrated against any labelled outcome.** An answer can score highly and be entirely false, because a fluent, self-consistent, well-cited, appropriately hedged falsehood passes every structural probe we run. We surface the score's decomposition so a reader can see which dimensions drove it, and we attach caveats to every card stating the limits of the audit within the card itself.

The verdict policy is deliberately conservative, on the view that over-trusting is the more costly error. A single high-severity structural failure suffices to reject regardless of the aggregate score. Our canonical canary illustrates this: the answer $9 \times 9 = 80$ yields $\mathrm{ECS} = 0.8143$, high, because the answer is short, self-consistent, appropriately scoped, and free of injection signals, while the verdict is `reject`, driven by one high-severity `arithmetic_sanity` failure. The score describes structure, the verdict describes acceptability, and they are not the same question.

5 The Lite Verifier

Lite runs seven probes over a bounded input of 6,000 characters for the question, 12,000 for the answer, and at most eight optional intents (Table 1).

Two design choices deserve comment. First, `arithmetic_sanity` uses an allowlist rather than a general expression evaluator, because a general evaluator over untrusted input is both a correctness and a security liability, and because a false negative is cheaper here than a false positive. Second, `prompt_injection` treats matched text strictly as data to be reported on, never as instruction to be followed: an auditor that executed the content it audits would be the attack surface rather than the defence.

Determinism. No probe calls a model or the network, so identical inputs yield identical outputs. We verified this against production rather than asserting it from source. Three identical requests to the live public endpoint returned byte-identical response bodies, with SHA-256 digest prefix `f33f8274`. We note the contrast with our own earlier six-tool MCP server, which is model-assisted and for which we make no determinism claim beyond canonical assembly and hashing (Section 12).

Table 1: The seven probes. All are model-free and network-free, and all are conservative: where a probe cannot reach a confident structural judgement, it passes. The deliberate limitation of each probe is given in italics. The system under-reports rather than over-reports, because an auditor that cries wolf gets disabled, and a disabled auditor has precision zero.

Probe	What it checks, and what it cannot
`claim_extraction`	Atomic claims within a fixed limit. *Cost bounded by design, not by answer length.*
`arithmetic_sanity`	Recomputes allowlisted expressions. *Silent outside the allowlist.*
`internal_contradiction`	Direct lexical and repeated-value conflicts. *Not inference-grade.*
`unsupported_certainty`	Certainty language without support. *A lexical signal only.*
`citation_verifiability`	Citation presence and transparency. *Citations are never authenticated.*
`fact_check_scope`	Answers overstating what a non-web audit establishes. *Scope only.*
`prompt_injection`	Instruction-like text, held inert. *A signal, never containment.*

6 The Public Projection

The public MCP projection is a deliberate subtraction. It omits the question, the answer, extracted excerpts, verifier evidence, free-form rationale, timestamps, audit and log identifiers, input hashes, and request or session metadata. What survives is the verdict, the score, counts, fixed-category findings, per-probe outcomes, and caveats: enough to act on, not enough to reconstruct the input.

We verified this behaviourally rather than by inspection. We issued a live production request with unique sentinel tokens embedded in both the question and the answer, and confirmed that neither token appeared anywhere in the response, and that none of `log_id`, `inputs_hash`, `generated_at`, `timestamp`, or `excerpt` was present. We recommend this test generally: projection policies are easy to state, easy to implement correctly once, and easy to regress silently when a field is added upstream.

What we do not claim. Platform providers and the infrastructure host receive and process request traffic. We therefore make no claim of "zero data collection," "nothing leaves the device," "no third parties," or end-to-end encryption, and we regard such claims in comparable systems as generally unsupportable. Our retention posture is bounded, not absent: raw content is not persisted to a database or to files by the gateway; content and results remain transiently in memory for up to five minutes while work and delivery complete; completed event identifiers are retained up to 24 hours for replay protection; and rate-control identifiers persist up to ten minutes.

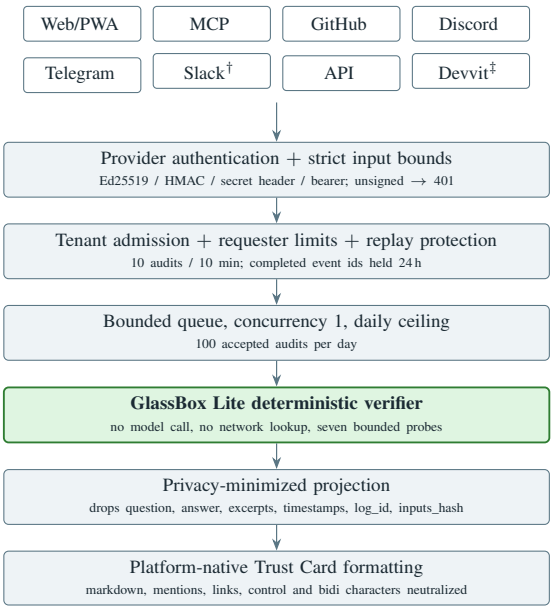

Figure 1: The GlassBox request path. A single deterministic verifier sits behind a uniform admission, bounding, and projection pipeline; adapters vary only in authentication and rendering. †Slack is an allowlisted single-workspace pilot. ‡Reddit Devvit is blocked pending outbound fetch-domain approval (Section 14).

7 Adapter Architecture

One verification core sits behind a uniform pipeline (Figure 1). Adapters differ only at the ends: how a provider authenticates, and how a result is rendered. Everything between is shared, which is what makes eight surfaces maintainable by one developer.

The design tension is that providers disagree about nearly everything. Discord signs with Ed25519 and expires interaction tokens on a deadline. Slack and GitHub sign with HMAC. Telegram uses a secret header. MCP clients speak JSON-RPC over Streamable HTTP [17]. Reddit Devvit requires an approved outbound-fetch allowlist. The gateway normalises these into a single admission decision, and stops delivery before a provider's token expires rather than emitting a late result.

8 Authentication and Abuse Controls

Every provider route validates provider-native authentication against the preserved raw request body and *fails closed*. Wrong events, missing delivery identifiers, unsigned requests, and forged requests are rejected before any verification work is queued.

Admission is explicit. Public availability is per-platform rather than global: the deployed configuration opens a selected subset and leaves the remainder allowlisted. Rate limits run *before* verifier invocation, at ten audits per ten minutes per requester, under a global ceiling of 100 accepted audits per day, with verifier concurrency pinned at one. Completed event identifiers are reserved through delivery and released on delivery failure, so a provider retry after a genuine failure still succeeds while an in-flight retry does not produce a duplicate reply.

Credential handling is minimal by construction. The MCP child process receives an explicit environment allowlist rather than platform credentials; marketplace payloads and OAuth tokens are neither logged nor persisted; a temporary OAuth token is revoked immediately after use; and OAuth state is signed, one-time, plan-bound, and expires after ten minutes.

Output is neutralized before delivery: markdown, mentions, links, control characters, and bidirectional characters are defanged, Discord mentions are disabled, and Slack link unfurling is off. An auditor that rendered hostile submitted content into a privileged channel would itself become the attack.

Table 2: The evidence scale used throughout this paper. Every claim we make is tagged with one of these levels, and Figure 2 places each deployed surface on it.

Level	Meaning
L4	Public production end-to-end: a real request traversed a public surface and produced a captured, publicly viewable result.
L3	Production signed or configured canary: an operator-initiated request against the live production route, correctly authenticated or deliberately forged.
L2	Automated test, local or continuous integration, including adversarial tests.
L1	Implementation and configuration present, not exercised end to end.
L0	Proposed or future work.

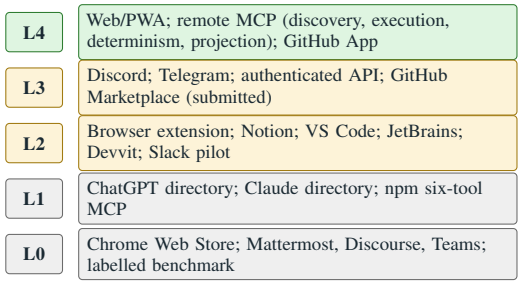

Figure 2: GlassBox surfaces on the evidence scale at the evaluated commit. Reporting the lower rows is the point: a paper that promotes every row to "available" destroys the information a reader needs in order to judge it.

9 Five Levels of Evidence

We found no adequate vocabulary for what we needed to report, so we adopted one (Table 2). The alternative is a single word, "tested," spanning an enormous range of epistemic strength: a system with a green test suite and a system with a publicly viewable production result are not in the same evidentiary position.

We answer no accuracy question at any level, because that requires a labelled benchmark we have not built (Section 18).

Table 3: Test coverage at the evaluated commit. All suites except JetBrains were re-executed locally during preparation of this paper; JetBrains is a Gradle project and rests on continuous-integration evidence.

Suite	Tests	Evidence
Gateway	69	re-run locally
Notion (native)	12	re-run locally
Browser extension	10	re-run locally
Reddit Devvit	9	re-run locally
VS Code	5	re-run locally
JetBrains	3	CI job green
Platform total	**108**	

Table 4: Fail-closed behaviour, probed against live production routes during preparation of this paper. No route admitted an unsigned or unauthenticated request.

Route	Probe	Status
`/discord/interactions`	unsigned	401
`/github/webhook`	unsigned	401
`/telegram/webhook`	no secret	401
`/slack/commands`	unsigned	401
`/slack/interactions`	unsigned	401
`/github/marketplace`	unsigned	401
`/api/v1/verify`	no bearer	401
`/github/marketplace/setup`	no plan	400

10 Adversarial Testing

At the evaluated commit, 108 automated tests pass across six heterogeneous suites (Table 3). This is the platform-layer total and not the repository's whole test count.

The gateway suite is weighted toward adversarial and operational behaviour rather than happy paths: forged and unsigned requests, wrong events, missing delivery identifiers, in-flight retry duplication, rate limits ahead of invocation, the global ceiling, stalled verification with transport reset, worker-slot release when a reset never settles, event reservation held through delivery and released on failure, and delivery halted before interaction-token expiry.

Twelve continuous-integration jobs succeed on the merged main commit: the gateway, Devvit, browser extension, Notion, VS Code, and JetBrains suites; TypeScript strict mode with an MCP smoke test; Python package build, install, and import on 3.10, 3.11, 3.12, and 3.13; and a cross-language audit-hash determinism assertion. Production dependency audits report zero *known* vulnerabilities at the configured threshold, which is not a penetration test, not static analysis, and not a certification. Continuous integration also emits Node 20 and transitive dependency deprecation warnings, which we do not suppress.

We additionally probed the live production routes directly. Every provider route rejected unsigned or unauthenticated requests (Table 4).

Table 5: GBSA-1 micro-averaged results. The middle row is reported only to expose the size of the tuning effect; the held-out row is the capability estimate.

Split	Precision	Recall	F1
Development, before repair (n=112)	0.975	0.812	0.886
Development, after repair (n=112)	1.000	1.000	1.000
Held-out, after repair (n=75)	**1.000**	**0.639**	**0.780**

11 Probe-Level Evaluation

The tests in Section 10 establish that the system behaves as specified. They do not establish that the probes catch what they are meant to catch. For that we built GBSA-1, a constructed benchmark of 187 items across six strata. Because Lite performs no model inference, the benchmark costs nothing to run and reruns byte identically; this is a direct consequence of the design choice under study.

Construction. Labels are fixed *by construction* rather than by post-hoc annotation: an item is generated into a stratum whose defining property determines its label. The arithmetic stratum has objective ground truth and involves no human judgement at all. Items were written from the published probe semantics of Table 1 and deliberately not from the implementation's regular expressions, since a benchmark derived from the matcher would only prove that the matcher matches itself. Every stratum carries near-miss negatives: surface-similar items on which the probe must stay silent.

Splits. A development split (112 items) was used to locate defects, and five were found and repaired. Repairing the implementation against that split makes its post-repair score tuned on the test set, so we do not quote it as a capability estimate. A held-out split (75 items) was written after the repairs, using surface forms deliberately excluded from every pattern that was touched. Table 5 reports all three measurements so the effect of tuning is visible rather than hidden.

The probes split into two classes. Table 6 gives the held-out result per probe, and the pattern is sharp. Probes that *compute* a property are indifferent to phrasing: `arithmetic_sanity` recomputes the expression and holds at 1.000 recall on unseen framings, and `internal_contradiction` tests negation polarity over normalised token overlap, which is structural rather than lexical, and holds at 0.833. Probes that *match a vocabulary* do not generalise: against phrasings outside their lexicon ("irrefutably", "categorically", "pay no attention to anything stated before this line") recall falls to between 0.000 and 0.333.

The repair history makes the point sharper than the scores alone. Broadening the three lexical vocabularies took development recall from 0.812 to 1.000 and left held-out recall for those same probes at or near zero. **Vocabulary repair buys development score, not capability.** We take this as a bound on the zero-inference design rather than a list of defects: a model-free auditor can be exact where the audited property is computable, and is only as broad as its lexicon where it is

Table 6: Held-out split, per probe. Precision is 1.000 wherever the probe fired at all. Recall tracks whether the probe computes a property or matches a lexicon.

Probe	Kind	P	R	F1
`arithmetic_sanity`	computed	1.000	**1.000**	1.000
`internal_contradiction`	structural	1.000	**0.833**	0.909
`citation_verifiability`	lexical	1.000	**0.333**	0.500
`unsupported_certainty`	lexical	n/a	**0.000**	0.000
`prompt_injection`	lexical	n/a	**0.000**	0.000
Micro-average		1.000	0.639	0.780

not. That is the honest price of removing inference, and it is the cost a model-based judge pays money to avoid.

Precision is where the design wins. There were **zero false positives across all 187 items**, including 14 benign clean controls and 34 near-miss negatives written to be surface-similar to positives ("The installation instructions say to ignore the optional dependencies"; "The error rate is 2 percent. It was 11 percent last quarter"). On the held-out split no probe fired outside its own stratum. For an auditor intended to run by default rather than on a sampled subset, precision is the property that decides whether it stays switched on, and precision is the property that holds.

Determinism at scale. Both splits were run three complete times. All passes were byte-identical. This is a stronger determinism result than the three-call production canary of Section 5: 187 distinct inputs, three full passes, identical digests.

What this does not measure. Items are constructed, not sampled from any real answer distribution, so nothing here says how often these failure modes occur in practice. Labels are fixed by construction and carry no inter-annotator agreement. Class balance is by design. No latency, cost, or throughput is measured. And no probe, at any recall, speaks to whether an answer is true.

12 Cross-Language Reproducibility

The repository's earlier six-tool MCP server carries a canonical audit-record construction: recursively sorted-key JSON serialization, SHA-256 hashing, and deliberate exclusion of the generation timestamp from the hash, so identical inputs and identical engine outputs collapse to the same audit identifier. Continuous integration asserts that a JavaScript run and an installed Python client both reproduce the reference identifier `glassbox-85cc09903bd4b3f8022a4087` byte for byte.

We state the scope of this result precisely, because it is easy to overclaim. The demonstration begins from *prebuilt* claim, red-team, and ECS parts. It establishes that canonical assembly, verdict policy, serialization, and hashing are stable across a language boundary. It does *not* establish that unconstrained model analysis is deterministic, and it concerns the model-assisted MCP rather than Lite. Lite's determinism is a separate and stronger result, established directly against production in Section 5.

13 Deployment Case Studies

GitHub App (L4). A public GitHub App receives issue-comment events, verifies HMAC signatures, and replies as a bot. Our canary submitted deliberately incorrect arithmetic through a real issue comment; GlassBox replied as `glassbox-by-aura[bot]` with a REJECT Trust Card (ECS 81.4 %, weakest dimension arithmetic integrity) approximately three seconds later. The comment is publicly viewable. This is our strongest public evidence, and it is a single operator-initiated canary rather than a user study.

Remote MCP (L4). The public endpoint answers `tools/list` with a single privacy-minimized tool and executes `tools/call` over Streamable HTTP. Our canary question "What is 9 multiplied by 9?" with answer "$9 \times 9 = 80$" returns `verdict=reject`, `score=0.8143`, one high-severity `arithmetic_sanity` finding, and six passing probes. Re-executed during preparation of this paper, it reproduced exactly.

Discord and Telegram (L3). Both are publicly registered, connected to authenticated production routes, and verified to reject unsigned requests. **We preserved no real-user result transcript for either.** They are reported as publicly registered and live-configured, not as production end-to-end proven, and closing this gap is our second-priority experiment (Section 18). Table 7 gives the full matrix.

14 Negative Results

We report these as findings rather than apologies, because they are the transferable part of the work.

Public registration is not observed use. Discord and Telegram are installable and authenticated, and we still cannot show a real user receiving a result. The distance between "installable" and "observed working" is invisible in most deployment claims and is, in our experience, the most commonly overstated step.

A distribution gate can be architectural rather than functional. Our Slack adapter works: signed commands, visibility control, response-URL delivery, and modal flows all pass. Public Slack distribution nonetheless requires multi-workspace OAuth, encrypted token storage, uninstall and deletion processing, eligibility confirmation, and an active-workspace threshold. The remaining work is not the feature, it is the tenancy model. Slack therefore remains a deliberately allowlisted single-workspace pilot.

An outbound-domain allowlist can block a correct consent flow. Our Reddit Devvit app reached explicit consent, with menu shown, consent dialog accepted, and selection validated, then initiated the gateway call, which failed at the network stage while the gateway itself was healthy and serving other surfaces. The fetch-domain approval remains pending, which is the leading explanation. We state this as an inference and not as a confirmed platform determination, and no Trust Card was produced.

Directory eligibility is independent of protocol correctness. Our MCP endpoint is demonstrably correct and publicly usable by compatible clients. Publication in the ChatGPT or Claude directories nonetheless depends on submission tokens and, in one case, on holding an eligible organisation tier. A zero-cost individual developer can build a conforming endpoint and remain structurally ineligible for the directory that would make it discoverable.

A free listing can still require commercial data. Our GitHub Marketplace listing is a $0 plan with no paid tier and no trial. A real free install nonetheless reached a $0.00 order page requesting a private billing address. We entered none. The listing is submitted and pending publication review; at the time of writing the expected public listing URL returns HTTP 404, and we do not describe it as published, listed, or searchable.

Table 7: Platform availability at the evaluated commit. What is established appears in roman; *what is not established appears in italics*, and is the column that matters. Rows are ordered by evidence level rather than by prominence.

Surface	Lv.	Established, and what is not
Web/PWA	**L4**	Publicly reachable and serving. *No captured third-party session.*
Remote MCP	**L4**	Tool discovery, execution, byte-identical determinism, projection minimality. *Not in any public directory.*
GitHub App	**L4**	Bot-authored Trust Card on a public issue. *Not a Marketplace listing.*
Authenticated API	**L3**	Unauthenticated request rejected. *Not an open endpoint.*
Discord	**L3**	Installable; unsigned rejected; adapter tests pass. *No real-user transcript; not provider-verified.*
Telegram	**L3**	Registered bot; unsigned rejected; webhook contract tested. *No real-user transcript.*
GitHub Marketplace	**L3**	Submitted; signed canaries idempotent; setup validation. *Listing returns 404; not published, listed, or searchable.*
Browser ext., Notion, VS Code, JetBrains	**L2**	Suites pass; CI jobs green. *Not in any extension or plugin marketplace.*
Slack	**L2**	Signed commands, visibility, modal flows tested. *Absent from public platforms; no multi-workspace OAuth.*
Reddit Devvit	**L2**	Consent flow reached; suite passes. *Public review and fetch domain pending; no Trust Card produced.*
ChatGPT and Claude directories	**L1**	Endpoint technically compatible. *No captured UI end-to-end; no directory listing.*
Chrome Web Store	**L0**	*Not pursued; no payment was made.*

Free-tier cold starts conflict with acknowledgement deadlines. The host instance can sleep and cold-start, while providers such as Discord expire interaction tokens on a fixed deadline. Our resolution is to stop delivery before token expiry rather than emit a late result, which is a mitigation rather than a solution, and we make no guaranteed-latency claim.

15 Related Work

GlassBox sits beside four lines of work without competing directly with any of them.

Guardrails and output-level classifiers. NeMo Guardrails [19] enforces programmable dialogue rails; Llama Guard [9] classifies prompts and responses against a safety taxonomy; LlamaFirewall [6] layers guardrails for agent security. These systems interpose before release, which GlassBox does not, and they generally require model inference, which GlassBox does not. GlassBox differs in decomposing to claim level, in exposing a per-probe decomposition rather than a category label, and in operating under deployer-supplied expectations rather than a fixed taxonomy.

Runtime verification of agents. Classical runtime verification monitors executions against formal specifications [5], and shielding synthesises a corrective layer that prevents unsafe actions [2]. Recent work carries this to language agents. C-Trace [12] expresses regulatory requirements as policy predicates over agent execution traces and *rejects* non-compliant tool invocations at runtime. AgentGuard [13] builds an online Markov model of agent behaviour to provide quantitative assurance. Zhou [23] derives capability-integrity, behavioural-verifiability, and interaction-auditability requirements and proves structural results about them. This line of work is more powerful than ours along the dimension that matters most for agents: it operates over trajectories and it prevents. **We do not claim this position.** GlassBox verifies a completed answer, does not intervene, and has no formal specification language. Barma et al. [4] do perform pre-actuation blocking against temporal-logic invariants, but in a simulated cyber-physical setting, and those properties do not transfer to the system described here.

Hallucination detection. Surveys [10, 8] and benchmarks such as TruthfulQA [15], HaluEval [14], and FEVER [20] target factuality, which GlassBox explicitly does not address. The closest methodological relative is SelfCheckGPT [16], which is likewise resource-light and black-box, but achieves this by sampling the model repeatedly; it is therefore non-deterministic and still incurs inference cost. GlassBox occupies the strictly zero-inference corner, and pays for it by abandoning factuality entirely.

Specification-driven alignment and assurance. Constitutional AI [3] uses an explicit written specification at training time; GlassBox applies deployer-supplied expectations at audit time, with no training involvement. Risk-management frameworks [1] motivate auditable records, and our canoni-

cal audit construction is a modest instance; our contribution is less the hashing scheme than the observation that a fully deterministic verifier makes such records exact rather than approximate.

We do not claim to be first, and we do not claim an unoccupied niche. Our claim is narrower: the zero-inference corner of this design space is under-explored, and it buys deployment properties the model-based corner cannot.

16 Privacy and Ethics

GlassBox is opt-in and never audits ambient conversation. Results on public surfaces are content-free by projection. The system charges no fee and accepts no payments.

We are deliberate about what we do not claim. Traffic is processed by platform providers and by the infrastructure host, so absolute-privacy claims would be false. Retention is bounded rather than absent, with exact bounds in Section 6. A verification tool making unsupportable privacy claims is a worse failure than one making modest accurate ones, since the former asks users to trust it precisely where it should not be trusted.

17 Limitations

Method. GlassBox audits structure, not truth, and a well-formed falsehood passes. ECS is structural and uncalibrated. Contradiction detection is lexical and misses semantic conflict. Arithmetic support is allowlisted, so unsupported mathematics is silently out of scope. Prompt-injection signalling is a signal, not containment. Inputs are bounded, so long-document auditing is out of scope. The three lexical probes have near-zero recall on phrasings outside their vocabularies (Section 11), which is a property of the design and not a bug that further vocabulary work will fix.

Evidence. Our **L4** results are single operator-initiated canaries, not a user study. The GBSA-1 items are constructed rather than sampled from real traffic, single-author, and labelled by construction, so they carry no inter-annotator agreement and no claim about natural frequency. Discord and Telegram lack preserved user transcripts. Cross-language determinism begins from prebuilt parts and concerns the model-assisted MCP. Dependency audits report zero known vulnerabilities at a configured threshold and are not a security assessment. The Reddit diagnosis is an inference. External approvals are outside our control and have no committed timeline.

Threats to validity. *Construct:* we do not measure trustworthiness; we measure determinism, projection minimality, and fail-closed behaviour, and the "Trust Card" framing must not be read as a validated trust judgement. *Internal:* canary inputs were chosen to exercise a known probe and demonstrate reachability, not detector quality. *External:* one operator, one deployment, one free-tier host, with multi-tenant behaviour untested at concurrency one and a 100-per-day ceiling. *Reproducibility:* live canaries depend on a running instance that may drift from the pinned commit; the commit, release, and CI run are the stable anchors.

18 Future Work

GBSA-1 (Section 11) covers six strata with constructed items. The clearest remaining gap is a benchmark sampled from real answer distributions rather than written, with blinded multi-annotator labels. The strata we would extend are: correct and incorrect allowlisted arithmetic; direct contradictions with non-contradictory lexical controls; calibrated uncertainty versus unsupported certainty; citation-present, citation-absent, and unverifiable-source cases; prompt-injection and hostile-markup inputs; refusals and safe non-answers; and Unicode, control-character, and bidirectional-text attacks, the last doubling as a projection-regression set. Labels should be preregistered before any run, with blinded annotation by at least two annotators and a reported agreement statistic, yielding per-probe precision, recall, and F_1 with bootstrap confidence intervals, ablations by probe, and baseline comparison against an output-level classifier. **None of that sampled-and-annotated study has been run;** what is reported here is the constructed benchmark only.

Two further priorities follow. Capturing one real-user end-to-end result each on Discord and Telegram would move two surfaces from **L3** to **L4** and close our largest evidence gap. A cold-start latency and acknowledgement-deadline study would convert Section 14's final finding from an observation into a measurement.

19 Conclusion

We built an answer auditor permitted no model inference, and found the constraint bought more than it cost. What we gave up is the ability to assess truth, and we are explicit throughout that we cannot. What we gained is determinism, verified byte for byte against a live public endpoint, which propagates into idempotency, replay suppression, and exact audit records in a way a sampled verdict cannot, and precision that held at 1.000 across every item we tested. What we also found is the shape of the cost: probes that compute a property survive unseen phrasing, probes that match a vocabulary do not, and no amount of vocabulary work changes that. The glass box goes around the black box, not through it.

The larger lesson concerns deployment reporting. Running one verifier behind eight platforms taught us that the interesting failures are not in the detector. They are outbound-domain allowlists, tenancy models, acknowledgement deadlines, directory eligibility rules, and billing-profile requirements attached to free listings. These are rarely reported, and they determine whether a working system is a usable one. We have reported ours against an explicit five-level scale, including the levels we would rather not occupy, and we encourage the same discipline from others.

Artifact and Reproducibility

Source, tests, and continuous-integration configuration are public. The evaluated commit, tagged release, main-branch CI run, exact reproduction commands, per-suite results, and the full claim-to-evidence ledger are recorded in the repository under `research/`. Live canary invocations and their outputs are listed verbatim in `research/reproducibility.md`.

References

[1] Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, National Institute of Standards and Technology, 2023.

[2] Mohammed Alshiekh, Roderick Bloem, Rüdiger Ehlers, Bettina Könighofer, Scott Niekum, and Ufuk Topcu. Safe reinforcement learning via shielding. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI-18)*, pages 2669–2678, 2018.

[3] Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

[4] Karthik Barma, Anil Sanneboyina, and V. C. Premchand Yadav. Glass box at orbit: A constitutional AI verification framework for trustworthy autonomous CubeSat intelligence. *arXiv preprint arXiv:2606.02967*, 2026.

[5] Ezio Bartocci, Yliès Falcone, Adrian Francalanza, and Giles Reger. Introduction to runtime verification. In *Lectures on Runtime Verification*. Springer, 2018.

[6] Sahana Chennabasappa, Cyrus Nikolaidis, Daniel Song, David Molnar, and Stephanie Ding. LlamaFirewall: An open source guardrail system for building secure AI agents. *arXiv preprint arXiv:2505.03574*, 2025.

[7] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023.

[8] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, et al. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint arXiv:2311.05232*, 2023.

[9] Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. Llama Guard: LLM-based input-output safeguard for human-AI conversations. *arXiv preprint arXiv:2312.06674*, 2023.

[10] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12), 2023.

[11] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.

[12] Nafiseh Kahani, Masoud Barati, and Diana Addae. Runtime compliance verification for AI agents. *arXiv preprint arXiv:2606.19242*, 2026. Introduces C-Trace.

[13] Roham Koohestani. AgentGuard: Runtime verification of AI agents. *arXiv preprint arXiv:2509.23864*, 2025.

[14] Junyi Li, Xiaoxue Cheng, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. HaluEval: A large-scale hallucination evaluation benchmark for large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[15] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.

[16] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.

[17] Model Context Protocol Contributors. Model context protocol specification. `https://modelcontextprotocol.io/specification`, 2025. Accessed 2026-08-11.

[18] Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527*, 2022.

[19] Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. NeMo Guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2023.

[20] James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. FEVER: A large-scale dataset for fact extraction and VERification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2018.

[21] Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language models don't always

say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[22] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[23] Ziling Zhou. Governing dynamic capabilities: Cryptographic binding and reproducibility verification for AI agent tool use. *arXiv preprint arXiv:2603.14332*, 2026.